DOCUMENTO MAESTRO / ESPECIFICACIÓN FUNCIONAL Y TÉCNICA

NEXUS AI

Plataforma de gestión empresarial con inteligencia artificial aplicada

DE LA IDEA A UN PRODUCTO DEMOSTRABLE

Especificación lista para entregar a ChatGPT Work como fuente de requisitos, prioridades, criterios de aceptación y pautas de implementación. El sistema se diseña para destacar habilidades de desarrollo full stack, arquitectura, seguridad, bases de datos y análisis con IA, bajo una política de costo cero.

Versión	1.0 · Documento base de ejecución
Fecha	22 de octubre de 2026
Propósito	Proyecto principal de portafolio profesional
Restricción rectora	S/0 en servicios externos; no compras ni suscripciones

Estado del documento: propuesta estructurada lista para implementación incremental

Resumen ejecutivo

NEXUS AI será una aplicación web de gestión empresarial orientada a pequeños negocios, implementada como proyecto demostrable de ingeniería de software. Integrará un frontend moderno en React y TypeScript, una API principal en Java con Spring Boot, una base de datos PostgreSQL y un servicio Python para análisis de datos y predicciones sencillas. El principal objetivo no es acumular herramientas, sino evidenciar que cada tecnología resuelve un problema claro, puede probarse y está documentada.

La primera entrega será un producto mínimo viable (MVP) con usuarios, roles, clientes, productos, movimientos de inventario, ventas, panel de indicadores y datos de demostración. Posteriormente se incorporará un módulo de inteligencia artificial con análisis reproducibles, y solo después funciones avanzadas como tareas, notificaciones y exportaciones.

DECISIÓN DE PRODUCTO Primero funcionalidad sólida de extremo a extremo; después mejoras visuales y capacidades de IA. Se prohíbe sustituir módulos no implementados por interfaces que aparenten funcionar.

Resultados esperados

- Repositorio organizado con historial de cambios y código ejecutable.
- Aplicación funcional en local, con instrucciones reproducibles y datos ficticios.
- Demo pública estática o full stack cuando exista una opción realmente gratuita y sostenible; la demo no dependerá de una suscripción.
- Pruebas automatizadas, documentación de arquitectura, casos de uso, capturas y una presentación técnica para reclutadores.
- Módulo Python que realice cálculos analíticos reales, con métricas, limitaciones y evaluación explícitas.

Índice del documento

01 · Identidad, visión y objetivos
02 · Alcance, prioridades y restricciones
03 · Usuarios, permisos y procesos
04 · Requisitos funcionales detallados
05 · Experiencia de usuario y diseño visual
06 · Arquitectura y tecnologías
07 · Modelo de datos y reglas de negocio
08 · Contratos de API e integraciones
09 · Inteligencia artificial y evaluación
10 · Seguridad, privacidad y calidad
11 · Pruebas y criterios de aceptación
12 · Estrategia de costo cero y despliegue
13 · Roadmap, entregables y puertas de calidad
14 · Entrega profesional para portafolio
15 · Instrucciones operativas para ChatGPT Work
Anexos · Checklist, glosario y fuentes oficiales

01 · Identidad, visión y objetivos

1.1. Problema y oportunidad

Muchos negocios pequeños administran ventas, inventario y clientes mediante archivos aislados, mensajes y registros manuales. Esto dificulta conocer el estado del negocio y tomar decisiones basadas en datos. NEXUS AI propone centralizar la operación en una herramienta comprensible y extensible que también permita extraer conclusiones útiles de la información histórica.

1.2. Propuesta de valor

Un usuario autorizado podrá registrar productos y clientes, procesar ventas, conocer existencias, consultar métricas y solicitar análisis de tendencias. El sistema aplicará permisos por rol, trazabilidad básica y validaciones de negocio para demostrar prácticas de ingeniería más allá de un CRUD.

1.3. Objetivos medibles

ID	Objetivo	Evidencia de cumplimiento
OBJ-01	Operación integral de ventas e inventario.	Registrar una venta y actualizar existencias e indicadores sin pasos manuales.
OBJ-02	Arquitectura full stack reproducible.	Levantar frontend, API, BD y servicio Python con documentación.
OBJ-03	Inteligencia artificial verificable.	Comparar predicciones contra un conjunto de prueba y mostrar su error.
OBJ-04	Seguridad y permisos reales.	Impedir acceso a funciones no autorizadas también desde la API.
OBJ-05	Presentación de alto nivel.	Demo, README, capturas, diagrama, pruebas y caso de estudio.

1.4. Público objetivo

Primario: evaluadores técnicos, empleadores y reclutadores que revisan el portafolio del desarrollador. Secundario: propietarios o administradores de pequeñas empresas que podrían validar los procesos del prototipo. El diseño inicial será de una sola empresa de demostración; no se implementará multitenencia real en el MVP.

02 · Alcance, prioridades y restricciones

2.1. Priorización de entregas

Nivel	Incluye	Criterio
P0 · Obligatorio	Autenticación, roles, clientes, productos, inventario, ventas, dashboard, PostgreSQL, API Java, diseño responsivo, pruebas esenciales.	Sin esto no existe un MVP funcional.
P1 · Diferenciador	Servicio Python, métricas descriptivas y pronóstico evaluable, exportación CSV, filtros avanzados, auditoría y documentación técnica.	Se implementa después de P0.
P2 · Expansión	Empleados, proyectos, tareas, avisos en tiempo real, reportes PDF, colas, caché, asistente conversacional local.	Solo si P0/P1 están probados y estables.

2.2. Fuera de alcance inicial

- Cobros reales, pasarelas de pago, emisión fiscal oficial, integraciones bancarias y facturación tributaria.
- Datos personales reales, importación de clientes de una empresa sin consentimiento y credenciales compartidas.
- Microservicios adicionales, Kubernetes, aprendizaje profundo entrenado desde cero y modelos de lenguaje de pago.
- Multitenencia de clientes empresariales, aplicaciones móviles nativas, pagos recurrentes y operaciones de producción.
- Alta disponibilidad, servicio garantizado 24/7 o un compromiso de soporte comercial.

2.3. Supuestos de diseño

Idioma de interfaz: español. Moneda de referencia: sol peruano (PEN). Zona horaria de referencia: America/Lima. La información inicial será completamente ficticia. En la base de datos los importes monetarios se representarán con DECIMAL, nunca con punto flotante; las fechas se almacenarán con zona horaria o normalizadas de manera consistente.

REGLA DE AHORRO Priorizar instalación local, bibliotecas abiertas y cuotas gratuitas sin facturación. No solicitar tarjeta ni activar pruebas pagadas sin una decisión posterior y explícita del usuario.

03 · Usuarios, permisos y procesos

3.1. Perfiles

Rol	Capacidades	Limitaciones
Administrador	Configura usuarios, roles, catálogos, inventario, ventas y reportes.	Sin acceso a credenciales en texto claro.
Gestor	Administra productos, clientes, inventario y ventas.	No crea administradores ni modifica permisos.
Analista	Consulta indicadores, reportes y análisis de IA.	Sin edición de datos operativos.
Lector	Visualiza módulos autorizados y datos de demostración.	No crea, edita ni elimina.

3.2. Matriz mínima de autorizaciones

Recurso	Admin	Gestor	Analista	Lector
Usuarios y roles	CRUD	—	—	—
Productos y clientes	CRUD	CRUD	Lectura	Lectura
Inventario	Control	Control	Lectura	Lectura
Ventas	Control	Crear / ver	Lectura	Lectura
Dashboard y reportes	Ver	Ver	Ver	Ver
Analítica IA	Ver	Ver	Ver	Opcional

Estas reglas se deben implementar tanto en la interfaz como en la API. Ocultar botones no equivale a controlar permisos.

3.3. Historias de usuario críticas

1. Como administrador, quiero crear un usuario con un rol limitado para controlar quién modifica información.
2. Como gestor, quiero registrar una venta de varios productos para que el stock y el total se actualicen automáticamente.
3. Como gestor, quiero conocer los productos por agotarse para reponerlos oportunamente.
4. Como analista, quiero filtrar ventas por fecha y categoría para identificar tendencias.
5. Como analista, quiero ver una predicción con su nivel de error y advertencias para no confundirla con una certeza.
6. Como evaluador, quiero entrar en modo demostración y comprobar funcionalidades sin configurar manualmente datos.

04 · Requisitos funcionales detallados

Cada requisito tiene un identificador estable, una prioridad y un comportamiento verificable. ChatGPT Work debe vincular tareas, pruebas y entregas con estos identificadores.

4.1. Acceso y administración de usuarios

ID / prioridad	Requisito	Criterio de aceptación
RF-01 · P0	Inicio y cierre de sesión con validación segura.	Credenciales erróneas no generan sesión; cierre invalida acceso.
RF-02 · P0	Gestión de roles y autorización por ruta.	Una llamada API no autorizada devuelve 403.
RF-03 · P0	Cuenta administradora y cuentas demo sembradas.	Existen al instalar datos de ejemplo, sin secretos en repositorio.
RF-04 · P1	Recuperación o reinicio administrativo de contraseña.	Token temporal o cambio controlado, sin exposición de contraseña.
RF-05 · P1	Perfil básico y cambio de contraseña.	Se pide la contraseña actual y se aplica política mínima.

4.2. Catálogo, clientes y proveedores

ID / prioridad	Requisito	Criterio de aceptación
RF-06 · P0	CRUD de productos con SKU, precio, costo, categoría, stock mínimo y estado.	SKU único; no admite importes negativos.
RF-07 · P0	Listado con búsqueda, filtrado, orden y paginación.	Se mantienen filtros en la URL cuando resulte útil.
RF-08 · P0	CRUD de clientes con datos mínimos y estado.	Validación de correo opcional; no guardar datos sensibles innecesarios.
RF-09 · P1	Registro de proveedores y asociación con productos.	Puede consultarse proveedor preferido por producto.
RF-10 · P1	Desactivación lógica de entidades utilizadas.	No se rompe el historial de ventas al desactivar.

4.3. Inventario y operaciones

ID / prioridad	Requisito	Criterio de aceptación
RF-11 · P0	Stock por producto derivado de movimientos válidos.	Cada entrada/salida produce un movimiento trazable.
RF-12 · P0	Ingreso y ajuste manual con motivo obligatorio.	Las existencias finales se recalculan sin valores negativos.
RF-13 · P0	Aviso de stock bajo.	Producto queda destacado si stock <= mínimo.
RF-14 · P0	Venta con líneas, cantidades, precios y total.	Operación atómica: venta y descuento de stock ocurren juntos.
RF-15 · P0	Anulación de venta con reversión controlada.	Se evita duplicar reversos; queda historial de anulación.
RF-16 · P1	Exportación CSV de movimientos y ventas.	Se respetan filtros y encabezados comprensibles.
RF-17 · P1	Registro de cambios relevantes.	Actor, fecha, entidad, acción y resultado disponibles para auditoría.

4.4. Dashboard y reportes

ID / prioridad	Requisito	Criterio de aceptación
RF-18 · P0	Indicadores: ventas del periodo, pedidos, ticket promedio y stock bajo.	Valores coinciden con consultas verificables de base de datos.
RF-19 · P0	Gráficas por fecha, categoría y productos destacados.	Estados de carga, sin datos y error definidos.
RF-20 · P0	Filtros por periodo de fechas.	Mismo intervalo produce mismo resultado en tabla y gráfico.
RF-21 · P1	Comparación de periodos y evolución histórica.	Expone fórmula y base de comparación.
RF-22 · P1	Descarga de reporte y ficha de resultados.	CSV sin fórmulas maliciosas; PDF opcional P2.

4.5. Inteligencia artificial, expansión y soporte

ID / prioridad	Requisito	Criterio de aceptación
RF-23 · P1	Servicio Python para estadísticas y pronóstico.	Entrega datos, método, fecha y métrica de error.
RF-24 · P1	Detección de anomalías o cambios bruscos.	Se explica regla/modelo y se muestran ejemplos.
RF-25 · P1	Recomendaciones de reposición basadas en datos.	Expone cálculos y evita inventar demanda.
RF-26 · P2	Gestión de empleados, proyectos y tareas.	Asignación, estado, prioridad y vencimiento persistidos.
RF-27 · P2	Notificaciones por eventos.	No duplica mensajes y respeta permisos.
RF-28 · P2	Asistente conversacional local, limitado a datos permitidos.	No ejecuta SQL arbitrario ni revela datos de otros roles.

4.6. Estados, errores y consistencia

Todos los formularios tendrán validación en cliente y servidor, mensajes de éxito o fallo claros, manejo de duplicados y confirmación antes de acciones destructivas. No se mostrarán cifras inventadas: cuando falten registros se informará que todavía no hay datos. Las operaciones críticas utilizarán transacciones y restricciones a nivel de base de datos.

05 · Experiencia de usuario y diseño visual

5.1. Dirección de diseño

Interfaz SaaS profesional, sobria y contemporánea: fondo oscuro opcional con neutros profundos, acentos azul/cian y jerarquía visual precisa. El aspecto debe ser apropiado para una demo empresarial, no parecer una plantilla genérica. Todo elemento ornamental estará subordinado a la comprensión de datos y la interacción. Debe existir modo claro u oscuro coherente, no una mezcla arbitraria.

5.2. Navegación y pantallas

Pantalla	Contenido mínimo	Acciones clave
Acceso	Inicio de sesión, validaciones, vista demo.	Entrar y cerrar sesión.
Inicio / dashboard	KPIs, gráfica temporal, alertas, acciones rápidas.	Cambiar intervalo y navegar a detalle.
Productos	Tabla, filtros, formulario y detalles.	Crear, editar, desactivar.
Inventario	Existencias, alertas, historial y ajuste.	Registrar movimiento.
Ventas	Tabla, detalle y creador de venta.	Agregar líneas, confirmar, anular.
Clientes	Listado, búsqueda y ficha.	CRUD controlado.
Informes e IA	Métricas, pronóstico, advertencias y exportación.	Filtrar, comparar y exportar.
Administración	Usuarios, roles y parámetros básicos.	Aplicar permisos.

5.3. Criterios de interfaz

- Responsive: escritorio 1440/1280 px, tableta ~768 px y móvil ~390 px como referencias de validación.
- Accesibilidad: navegación por teclado, foco visible, etiquetas asociadas, contraste legible y texto alternativo pertinente.
- Componentes reutilizables: tabla de datos, campo de formulario, modal, diálogo de confirmación, tarjeta KPI, aviso, toast y estado vacío.
- No abusar de animaciones: transiciones breves que respeten prefers-reduced-motion.
- Los errores deben explicar cómo recuperarse, evitando mensajes vagos como «Algo salió mal».
- Cargar datos solo cuando corresponde; mostrar skeletons o indicadores cuando una operación tarde.

5.4. Recorrido demostrativo de tres minutos

1. Acceder con una cuenta de prueba y presentar el dashboard inicial.
2. Crear un producto y modificar su stock mediante un movimiento trazable.
3. Registrar una venta y mostrar el descuento automático de inventario.
4. Volver al dashboard para comprobar la actualización de indicadores.
5. Abrir analítica, generar un pronóstico, revisar su explicación y descargar resultados.
6. Mostrar permisos: intentar una operación de administrador desde un rol lector y demostrar el bloqueo.

06 · Arquitectura y tecnologías

6.1. Principio arquitectónico

La primera versión usará un backend Java monolítico modular por dominios, una base de datos PostgreSQL y un servicio Python separado para analítica. Esta separación permite demostrar integración entre lenguajes sin introducir complejidad innecesaria. Las fronteras deben estar claras: Java controla autenticación y reglas de negocio; Python ejecuta procesos analíticos de datos autorizados; React consume interfaces bien definidas.

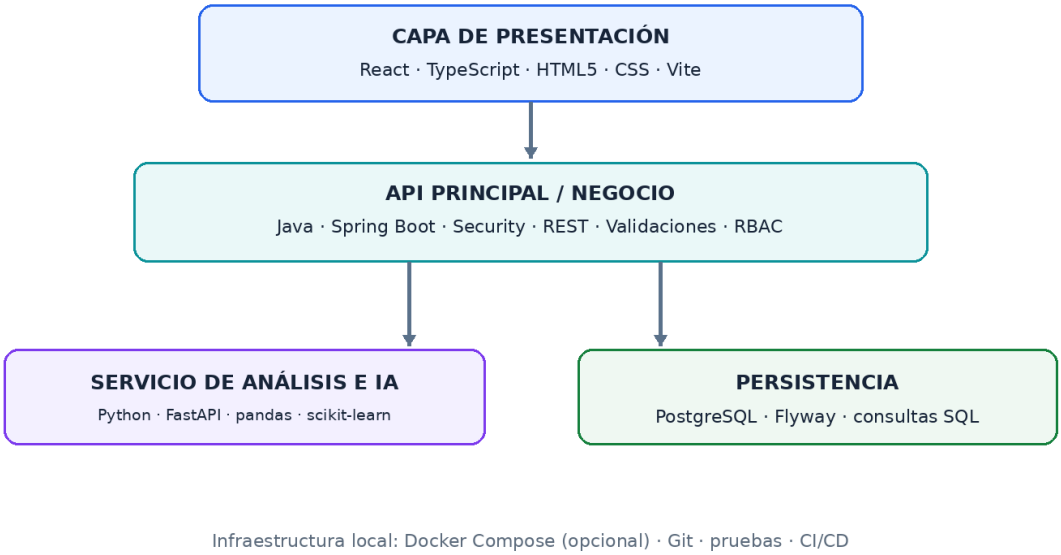


Figura 1. Arquitectura lógica de referencia para el MVP y la fase de IA.

6.2. Stack recomendado

Componente	Tecnología	Justificación / observación
Cliente web	React + TypeScript + Vite	Interfaz componible, tipado y compilación estática.
Estilos	CSS moderno y tokens; Tailwind opcional	Sistema visual propio, sin dependencia obligatoria de pago.
API principal	Java + Spring Boot + Spring Security	Dominios, transacciones, permisos y validación.
Persistencia	PostgreSQL + JPA/Hibernate + Flyway	Integridad referencial, consultas y migraciones.
IA y análisis	Python + FastAPI + pandas + scikit-learn	Cálculo analítico independiente y reproducible.
Pruebas	JUnit, Mockito, Testcontainers opcional, pytest, Vitest, Playwright opcional	Verificación automatizada en distintas capas.
Contenedores	Docker Compose opcional	Entorno local predecible; alternativa sin Docker documentada.
CI	GitHub Actions, dentro de cuotas	Lint, tests y build en cada cambio relevante.

6.3. Responsabilidades por servicio

- Frontend: navegación, presentación, estados de carga/error, validación de experiencia y consumo de API. Nunca es la fuente de verdad sobre permisos.
- Backend Java: reglas, persistencia, validaciones, transacciones, autenticación, RBAC, auditoría y API versionada.

- Servicio Python: recibe agregados/series temporales permitidos o peticiones validadas desde Java, procesa y devuelve respuestas estructuradas; no accede directamente a secretos de usuario.
- PostgreSQL: restricciones, índices, claves foráneas, integridad monetaria, migraciones y datos semilla.

6.4. Organización propuesta del repositorio

```
nexus-ai/
  apps/web/               # React + TypeScript
  services/api-java/      # Spring Boot (dominios y seguridad)
  services/analytics-python/ # FastAPI + modelos y pruebas
  infra/                  # compose, scripts y despliegue opcional
  docs/                   # ADR, diagramas, API, capturas, caso de estudio
  scripts/                # inicialización y validaciones locales
  .github/workflows/      # CI
  .env.example            # variables documentadas, sin valores secretos
  README.md               # guía principal
```

6.5. Integración y configuración

Usar API REST JSON con rutas versionadas (/api/v1). Configurar URLs y secretos por variables de entorno. El servicio Java debe imponer timeouts y tratamiento de fallos al llamar al servicio Python; la caída de analítica no debe bloquear una venta. Utilizar identificadores y códigos de error consistentes para depurar sin exponer datos internos.

07 · Modelo de datos y reglas de negocio

7.1. Entidades principales

Entidad	Campos esenciales	Relaciones / notas
users	id, display_name, email, password_hash, active, created_at	N:M con roles o rol simple inicial.
roles	id, code, description	ADMIN, MANAGER, ANALYST, VIEWER.
categories	id, name, active	1:N productos.
products	id, sku, name, category_id, cost, price, min_stock, active	SKU único, precios DECIMAL.
customers	id, name, email?, phone?, active	1:N ventas; datos mínimos.
suppliers · P1	id, name, contact?, active	Relación opcional con producto.
sales	id, code, customer_id?, total, status, occurred_at, created_by	1:N sales_items; usuario creador.
sales_items	id, sale_id, product_id, qty, unit_price, subtotal	Copia histórica del precio usado.
stock_movements	id, product_id, type, qty, reason, reference_id?, created_by, created_at	Entrada, salida, ajuste o reverso.
audit_events · P1	id, actor_id, action, entity, entity_id, created_at, metadata_safe	No registrar contraseñas ni tokens.
analytics_runs · P1	id, model_version, interval, metric, score?, created_at	Trazabilidad de inferencias.

7.2. Restricciones de integridad

- Claves primarias UUID o BIGINT elegidas de forma consistente; no mezclar estilos sin motivo.
- SKU, código de venta y correo de usuario únicos cuando corresponda.
- Precios/costos/totales con DECIMAL(12,2) o precisión acordada; cantidad entera estrictamente positiva.
- El total de una venta debe coincidir con la suma de subtotales y ajustes autorizados.
- Registrar ventas y movimientos dentro de una misma transacción. Bloquear carreras concurrentes en stock con estrategia justificada.
- Prohibir eliminaciones físicas de catálogos referenciados por ventas; usar estados de desactivación.
- Fechas coherentes y zonas horarias controladas; no asumir que la hora del navegador es la del servidor.

7.3. Regla crítica: registrar una venta

1. Validar usuario, rol, cliente opcional y productos activos.
2. Validar cantidades y obtener precio aplicable según la política definida.
3. Comprobar inventario con control de concurrencia en la transacción.
4. Crear cabecera y líneas de venta, registrar movimientos de salida y totalizar.
5. Confirmar la transacción únicamente si todas las operaciones tienen éxito.
6. Devolver ID, código, importe y nuevo estado; si ocurre una falla, deshacer todo.

7.4. Datos ficticios reproducibles

Crear un seed determinista con al menos cuatro usuarios de roles distintos, 30 productos, 8 categorías, 20 clientes, 3 meses o más de ventas sintéticas y casos intencionados de stock bajo. Los datos deberán ser creíbles pero claramente ficticios. El script debe poder reconstruir el entorno de demostración sin duplicaciones.

08 · Contratos de API e integraciones

8.1. Convenciones

Prefix /api/v1, respuestas JSON, nombres consistentes, paginación para listados, errores tipados y documentación OpenAPI. Aplicar validación de entrada y autorización antes de ejecutar lógica sensible. Definir una estrategia consistente de fecha/hora e importes.

8.2. Endpoints representativos

Método y recurso	Uso / control
POST /api/v1/auth/login	Iniciar sesión; aplicar protección ante intentos repetidos.
POST /api/v1/auth/logout	Cerrar sesión e invalidar credenciales cuando corresponda.
GET /api/v1/me	Identidad y permisos efectivos.
GET/POST /api/v1/products	Listar / crear productos; POST exige rol autorizado.
GET/PATCH /api/v1/products/{id}	Detalle / edición.
GET/POST /api/v1/customers	Listar / registrar clientes.
GET /api/v1/inventory/movements	Consultar movimientos con filtros.
POST /api/v1/inventory/adjustments	Registrar ajuste con motivo y auditoría.
GET/POST /api/v1/sales	Listar / crear venta transaccional.
POST /api/v1/sales/{id}/cancel	Anulación idempotente o con prevención de doble reversión.

Método y recurso	Uso / control
GET /api/v1/dashboard/summary	Indicadores filtrados por fecha.
POST /api/v1/analytics/forecast · P1	Solicitar pronóstico autorizado a Python.
GET /api/v1/reports/sales.csv · P1	Descarga CSV segura.

8.3. Ejemplo de creación de venta

```
POST /api/v1/sales
{
  "customerId": "uuid-opcional",
  "items": [
    { "productId": "uuid-producto-1", "quantity": 2 },
    { "productId": "uuid-producto-2", "quantity": 1 }
  ]
}

201 Created
{
  "id": "uuid-venta",
  "code": "V-2026-0001",
  "currency": "PEN",
  "total": "89.70",
  "status": "CONFIRMED"
}
```

El cliente no envía el precio como fuente de verdad: el servidor lo determina según sus reglas. Los identificadores y montos anteriores son solo ejemplos de contrato.

8.4. Catálogo de errores

HTTP	Cuándo se usa	Ejemplo de código
400	Formato o solicitud inválida.	VALIDATION_ERROR
401	Sin autenticación válida.	UNAUTHENTICATED
403	Sin permiso para la operación.	FORBIDDEN
404	Recurso inexistente o no visible.	NOT_FOUND
409	Conflicto de stock, duplicidad o estado.	STOCK_CONFLICT
422	Regla de negocio no satisfecha.	BUSINESS_RULE_VIOLATION
503	Servicio de analítica no disponible.	ANALYTICS_UNAVAILABLE

09 · Inteligencia artificial y evaluación

9.1. Propósito verificable

La IA debe apoyar decisiones con análisis reales y ser reproducible. No debe presentarse una heurística simple como si fuera un modelo complejo, ni inventar predicciones a partir de datos insuficientes. En P1 se desarrollará

un servicio Python que reciba series de ventas diarias o semanales, construya métricas y genere pronósticos con evaluación retrospectiva.

9.2. Funciones priorizadas

Función	Técnica inicial	Salida mostrada al usuario
Resumen descriptivo	pandas + agregaciones SQL validadas	Series, totales, variación entre periodos.
Pronóstico ventas	Baseline estacional / media móvil; modelo de scikit-learn si mejora.	Valor previsto, intervalo orientativo si se justifica, error MAE/MAPE cuando proceda.
Detección anomalías	Reglas estadísticas o IsolationForest si resulta apropiado.	Fecha, valor, motivo y advertencia de falsos positivos.
Reabastecimiento	Demanda histórica y stock actual.	Productos y cálculo de días estimados de cobertura.
Chat local · P2	Modelo abierto pequeño, únicamente si es viable.	Explicaciones sobre agregados autorizados, con límites explícitos.

9.3. Experimentos y métricas

Separar temporalmente entrenamiento y validación para evitar fuga de datos. Comparar los modelos con una línea base ingenua; reportar al menos MAE y, solo cuando tenga sentido matemático, MAPE o sMAPE. Guardar periodo, volumen de datos, fecha de evaluación y versión del método. Un modelo que no supera la línea base no se comercializará como mejora: se documentará el resultado y se conservará el baseline.

9.4. Reglas de honestidad del módulo IA

- Si hay pocos datos, mostrar «historial insuficiente» y no una curva convincente inventada.
- Presentar «estimación» y «margen de incertidumbre» sin prometer certeza o causalidad.
- No hacer recomendaciones que dependan de información inexistente o no verificable.
- La IA no tendrá permisos para registrar ventas, editar existencias ni modificar usuarios.
- No enviar información empresarial a APIs externas comerciales sin autorización explícita.
- Los modelos locales son opcionales y pueden ser demasiado lentos para el hardware disponible; el módulo predictivo clásico funciona sin chatbot.

9.5. Contrato mínimo de predicción

```
{
  "metric": "daily_sales_pen",
  "forecastHorizonDays": 7,
  "predictions": [{ "date": "2026-11-01", "value": 145.50 }],
  "method": "seasonal_naive_v1",
  "evaluation": { "mae": 28.40, "sampleSize": 60 },
  "warnings": ["Predicción orientativa; datos ficticios"]
}
```

10 · Seguridad, privacidad y calidad

10.1. Controles obligatorios

Área	Requisito técnico
Contraseñas	Hash robusto con algoritmo mantenido por Spring Security; nunca guardar texto claro.
Sesiones / tokens	Elegir y documentar un modelo seguro. Si se usan JWT: acceso corto, no persistir tokens sensibles en localStorage; renovación segura en fase posterior.
Autorización	RBAC en cada controlador/servicio, además de controles del frontend.
Entradas	Validación estricta de DTO, parámetros, formatos, longitudes y montos.
SQL	ORM y consultas parametrizadas; prohibidas concatenaciones inseguras.
CORS / CSRF	Configuración según el modelo real de sesión y orígenes autorizados; sin comodines innecesarios.
Secretos	Variables de entorno y .env.example; .env real excluido de Git.
Logs	Sin contraseñas, tokens, datos personales innecesarios o consultas sensibles.
Dependencias	Versiones compatibles y actualizadas, escaneo cuando sea viable.
Datos demo	Solo registros ficticios; acción visible de restablecimiento cuando corresponda.

10.2. Requisitos no funcionales

ID	Calidad esperada	Verificación propuesta
RNF-01	Interfaces funcionales en móvil, tableta y escritorio.	Pruebas visuales en anchos de referencia.
RNF-02	Operaciones comunes sin errores bajo carga demo.	20 clientes concurrentes en entorno local de prueba definido; informar hardware y resultados.
RNF-03	Tiempo de respuesta medido, no prometido globalmente.	Objetivo orientativo p95 < 500 ms para CRUD local con conjunto de prueba; excluye cold start.
RNF-04	Accesibilidad práctica.	Revisión teclado, foco, etiquetas, contraste y estados.
RNF-05	Recuperación de datos.	Seed reconstruible y exportación SQL documentada.
RNF-06	Mantenibilidad.	Separación por capas, estilo uniforme, pruebas y documentación.
RNF-07	Observabilidad mínima.	Logs estructurados y endpoints de salud sin información sensible.

10.3. Política de credenciales para demostración

Una cuenta demo jamás tendrá privilegios reales en servicios externos ni dará acceso a información auténtica. El sistema deberá usar credenciales de ejemplo rotables y datos reinicializables. Si se publica la demo, los permisos de escritura pueden deshabilitarse o limitarse a sesiones aisladas. No incluir una contraseña administrativa permanente en el README de un despliegue de acceso público.

11 · Pruebas y criterios de aceptación

11.1. Estrategia de pruebas

Nivel	Qué cubre	Herramientas orientativas
Unitarias	Cálculo de totales, reglas de stock, validadores y autorizaciones.	JUnit 5, Mockito y pytest.
Integración	Persistencia, migraciones, transacciones de venta y API.	Spring Boot Test; PostgreSQL de prueba.
Contrato	Esquemas de solicitud, respuesta y errores.	OpenAPI + tests automáticos.
Frontend	Componentes, estados y validaciones.	Vitest y Testing Library.
End to end	Login, venta, dashboard, permisos y analítica.	Playwright, si está disponible.
Seguridad	RBAC, credenciales, entradas maliciosas y secretos.	Pruebas negativas + revisión de dependencias.

11.2. Escenarios de aceptación imprescindibles

1. Venta correcta: con stock suficiente se crean venta, líneas y movimientos; el total y existencias quedan consistentes.
2. Venta fallida: con stock insuficiente no se crea ninguna parte de la venta.
3. Concurrencia: dos ventas simultáneas no pueden dejar inventario negativo.
4. Anulación: se restablece stock una sola vez y queda registrada la razón.
5. Autorización: lector no puede crear productos manipulando directamente la API.
6. Datos: el dashboard devuelve métricas iguales al reporte SQL de control.
7. IA: el pronóstico informa período de entrenamiento, versión, baseline y MAE.
8. Resiliencia: si Python está caído, las ventas funcionan y la interfaz de IA muestra error recuperable.
9. Instalación: otra persona clona el repositorio y reproduce el MVP siguiendo el README.
10. Calidad: el pipeline de pruebas y build termina sin errores conocidos antes de una entrega.

11.3. Definición de terminado (DoD) por funcionalidad

Una historia se considera terminada únicamente si cumple su criterio de aceptación, tiene validación y errores, pruebas pertinentes, autorización correcta, UI adecuada, actualización de documentación y revisión manual de su caso feliz y casos fallidos. No se marca como completa una característica cuya pantalla solo tiene botones ficticios.

12 · Estrategia de costo cero y despliegue

12.1. Política económica no negociable

PRESUPUESTO Costo externo objetivo: S/0. Prohibidas compras, dominios de pago, tarjetas añadidas, conversiones a planes premium, APIs con facturación por uso y servicios que requieran cobro para mantener el MVP.

12.2. Modalidades de entrega

Modalidad	Qué incluye	Ventajas y límites
A · Local, garantizada como objetivo	React + Java + PostgreSQL + Python ejecutándose en PC o entorno compatible.	Sin hosting de pago. Necesita levantar servicios para demo en vivo.
B · Demo visual pública	Frontend React publicado en GitHub Pages con datos semilla o API simulada, claramente identificado como demo.	Disponible por enlace sin backend; no aparentar persistencia real.
C · Full stack gratuita, opcional	Despliegue externo solo si un proveedor permite ejecutar todos los servicios y persistir datos sin facturar.	Depende de límites, disponibilidad y requisitos de alta; no bloquear el proyecto.

12.3. Precauciones con proveedores

GitHub Pages es una alternativa natural para el frontend estático, no para ejecutar Java/Python. Render ofrece servicios gratuitos con suspensión por inactividad y límites; su PostgreSQL gratuito tiene vencimiento a los 30 días, por lo que no constituye base estable de datos a largo plazo. Oracle Cloud publica recursos Always Free, pero su capacidad puede no estar disponible y su proceso de alta debe evaluarse sin aceptar cargos o tarjetas por defecto. Las condiciones cambian: verificar siempre las páginas oficiales antes de ejecutar cualquier despliegue.

12.4. Estrategia práctica de demo

- Entregar primero una instalación local documentada y plenamente operativa.
- Publicar una presentación estática clara en el portafolio con capturas, recorrido y enlace al repositorio.
- Ofrecer un frontend demo alojado gratuitamente con datos simulados, marcado como tal, si el hosting full stack no es viable.
- Preparar un video breve o GIF de funcionalidad real como prueba visual, sin datos personales.
- Crear scripts para sembrar o restaurar la base de datos local.
- No activar pagos automáticos; si una plataforma exige tarjeta, buscar alternativa o continuar sin ese despliegue.

13 · Roadmap, entregables y puertas de calidad

Las fases se ordenan por dependencias reales. ChatGPT Work deberá completar y verificar cada una antes de avanzar. Las estimaciones de esfuerzo no son compromisos de calendario: se ajustarán según herramientas, límites del entorno y pruebas.

Fase	Construcción	Salida que debe existir
F0 · Fundación	Monorepo, estilos base, proyecto Java, PostgreSQL, migraciones, CI inicial, documentación.	Arranque local comprobado; API health y frontend visibles.
F1 · Seguridad y catálogos	Usuarios, roles, login, productos, categorías y clientes.	CRUD seguro con formularios y pruebas.
F2 · Operación central	Movimientos, control de stock, ventas, anulaciones.	Flujo completo transaccional y pruebas de concurrencia.
F3 · Reportería	Dashboard, gráficos, filtros, consulta SQL validada.	KPIs correctos frente a seed.
F4 · IA aplicada	FastAPI, series históricas, baseline, predicción, evaluación.	Predicción real explicada y documentada.

Fase	Construcción	Salida que debe existir
F5 · Calidad y entrega	E2E, accesibilidad, hardening, documentación, capturas, despliegue viable.	README completo, demo y checklist aprobado.
F6 · Opcionales	Tareas, empleados, tiempo real, CSV/PDF, chatbot local.	Solo si F0–F5 están estables.

13.1. Entregables obligatorios por fase

- Código fuente legible con instrucciones de ejecución exactas.
- Archivo de variables de entorno de ejemplo y migraciones verificables.
- Pruebas ejecutadas y resumen de resultado, incluyendo fallos pendientes.
- Capturas o evidencia visual del módulo y breve explicación de lo implementado.
- Registro corto de decisiones de arquitectura (ADR) cuando haya alternativas importantes.
- Lista de cambios y próximos pasos, sin afirmar tareas no comprobadas.

13.2. Gestión de incidencias y deuda técnica

Mantener un backlog con prioridad, descripción, reproducción, impacto y criterio de cierre. Si una integración no se puede probar en el entorno, marcarla como pendiente y ofrecer una instrucción de verificación local. Evitar «trabaja en mi máquina» como único criterio de calidad.

14 · Entrega profesional para portafolio

14.1. Página de presentación del proyecto

Incluir nombre, problema, rol del desarrollador, tecnologías realmente utilizadas, retos técnicos, decisiones de diseño, capturas, demo y repositorio. Cada afirmación deberá corresponder a código visible o evidencia. No afirmar despliegue 24/7, IA avanzada o pruebas exhaustivas si no existen.

14.2. README recomendado

1. Descripción y problema que resuelve NEXUS AI.
2. Capturas de escritorio y móvil, con accesibilidad de imágenes.
3. Stack y justificación, sin enumerar herramientas que nunca se usaron.
4. Arquitectura y diagrama de servicios.
5. Requisitos para entorno local y pasos de instalación reproducibles.
6. Variables, migraciones, seed, usuarios de demostración y seguridad.
7. Comandos para ejecutar, probar, construir y restablecer datos.
8. API y contratos relevantes.
9. IA: método, evaluación, dataset, advertencias y límites.
10. Decisiones de ingeniería, aprendizajes, estado real y roadmap.

14.3. Evidencias de ingeniería de alto impacto

- Demostración visual del flujo completo de venta con actualización de inventario y KPIs.
- Captura de pruebas pasando y un caso de autorización rechazado por backend.
- Diagrama de arquitectura y modelo ER simplificado.
- Comparación cuantitativa del pronóstico frente al baseline.
- Pipeline CI reproducible y ejemplo de migración de esquema.

- Caso de estudio conciso: reto, enfoque, trade-offs, resultado y próximos pasos.

15 · Instrucciones operativas para ChatGPT Work

La siguiente sección está redactada para que pueda copiarse en una conversación de ChatGPT Work junto con este archivo Word. Define el contrato de ejecución y reduce el riesgo de que el agente construya una maqueta en vez de un producto funcional.

15.1. Prompt maestro para Work

```
Construye NEXUS AI a partir del documento Word adjunto.
Trata este documento como fuente principal de requisitos.

OBJETIVO:
Implementar un proyecto de portafolio full stack real y verificable,
priorizando F0 a F3 (MVP) antes del módulo Python (F4).

REGLAS:
1. No realizar compras ni activar servicios de pago; objetivo S/0.
2. No solicitar ni guardar tarjetas de pago ni secretos en el código.
3. No fingir funcionalidades, pruebas, datos persistidos o despliegues.
4. Usar React + TypeScript, Java/Spring Boot y PostgreSQL.
5. Añadir Python/FastAPI únicamente tras estabilizar el MVP.
6. Mantener un monorepo claro, código mantenible y README ejecutable.
7. Crear migraciones, seed ficticio, pruebas y errores coherentes.
8. Validar permisos en el backend, no solo en la interfaz.
9. Antes de tocar repositorios o cuentas existentes, confirmar alcance.
10. No hacer push, publicar ni cambiar configuraciones externas sin
    autorización explícita para esas acciones.

EJECUCIÓN:
- Inspecciona el entorno y detecta las herramientas disponibles.
- Presenta un plan corto de fases y dependencias.
- Implementa iterativamente, ejecutando build/test tras cada fase.
- Cuando una herramienta no exista, documenta alternativa comprobable.
- Deja evidencia del resultado, fallos y pasos exactos para continuar.
- Prioriza un flujo E2E de producto → inventario → venta → dashboard.

ENTREGA:
Código, migraciones, scripts, tests, documentación, capturas y
resumen de cambios. Indica qué RF y RNF cumple cada fase.
```

15.2. Formato esperado del informe de Work

Apartado	Información requerida
Implementado	RF completados, archivos relevantes y cómo probarlos.
Validación	Comandos realmente ejecutados y resultados.
Pendiente	RF sin cerrar, bloqueos y riesgos precisos.
Decisiones	Motivo de cada cambio arquitectónico importante.
Costo	Confirmación de que no se activaron cargos; herramientas gratuitas usadas.
Siguiente paso	Una prioridad concreta, dependencias y criterio de terminado.

15.3. Reglas frente a ambigüedades

Si surge una cuestión no definida, Work debe elegir la alternativa más simple compatible con la arquitectura, registrar la decisión y continuar si es reversible. Debe pedir confirmación solamente cuando falte una autorización para acceder, publicar, modificar datos existentes o asumir un posible costo. Los cambios de alcance se tratarán como propuestas, no como requisitos originales.

Anexo A · Checklist de aceptación del MVP

☐ El repositorio tiene estructura consistente y un README completo.
☐ El frontend inicia sin errores y navega sin enlaces rotos.
☐ La API Java arranca y se conecta a PostgreSQL.
☐ Las migraciones crean el esquema desde cero.
☐ El seed ficticio se instala de forma repetible.
☐ Los cuatro roles tienen permisos comprobados por endpoint.
☐ El formulario de login gestiona estados válidos e inválidos.
☐ Productos y clientes implementan CRUD y restricciones.
☐ Los movimientos de inventario son trazables.
☐ Una venta se registra en una sola transacción.
☐ El stock nunca queda negativo incluso con solicitudes concurrentes.
☐ Una venta anulada no revierte el stock dos veces.
☐ El dashboard coincide con los datos reales del backend.
☐ Hay diseño responsive y estados vacíos/de error.
☐ No hay secretos o tokens sensibles en el repositorio.
☐ Las pruebas principales se ejecutaron y constan en la entrega.
☐ El servicio Python ofrece al menos un análisis reproducible (fase P1).
☐ El módulo de pronóstico muestra línea base y error de evaluación.
☐ La demo pública, si existe, distingue datos simulados de datos persistidos.
☐ No se han activado pagos ni suscripciones para desarrollar o desplegar.

Anexo B · Convenciones y glosario

Término	Definición
MVP	Versión mínima que resuelve un flujo completo y verificable.
CRUD	Crear, leer, actualizar y eliminar o desactivar registros.
RBAC	Control de acceso basado en roles.
API REST	Interfaz HTTP para consultar o modificar recursos mediante contratos.
Seed	Conjunto reproducible de datos de ejemplo.
Transacción	Operaciones que se confirman o revierten de forma conjunta.
Baseline	Modelo simple de referencia para comparar predicciones.
MAE	Error absoluto medio; promedio de la magnitud del error.
CI/CD	Automatización de validación, construcción y eventual despliegue.
DoD	Definición de terminado utilizada para aceptar una tarea.
ADR	Registro corto de una decisión arquitectónica y sus consecuencias.

Anexo C · Fuentes oficiales y condiciones

Las referencias siguientes se incluyeron para revisar límites de plataformas y licencias antes de desplegar. Consultadas en octubre de 2026. Su inclusión no garantiza disponibilidad futura ni que un proveedor sea adecuado para todos los componentes del proyecto.

GitHub Docs · Creating a GitHub Pages site
https://docs.github.com/en/pages/getting-started-with-github-pages/creating-a-github-pages-site

Render Docs · Deploy for Free
https://render.com/docs/free

Oracle Cloud · Always Free Resources
https://docs.oracle.com/en-us/iaas/Content/FreeTier/freetier_topic-Always_Free_Resources.htm

Docker Docs · Docker Desktop license agreement
https://docs.docker.com/subscription-billing/desktop-license/

Spring Boot · Reference Documentation
https://docs.spring.io/spring-boot/reference/

FastAPI · Official documentation
https://fastapi.tiangolo.com/

PostgreSQL · Documentation
https://www.postgresql.org/docs/

scikit-learn · User guide
https://scikit-learn.org/stable/user_guide.html

Cierre y principio de ejecución

NEXUS AI deberá ser evaluado por la calidad y comprobabilidad de su ingeniería: un recorrido completo, datos consistentes, arquitectura explicable, IA honesta, seguridad básica y documentación clara. El criterio de éxito no es utilizar todas las tecnologías posibles, sino demostrar criterio técnico al utilizarlas donde aportan valor.